

Fed Mandate Performance Data Fetching

Mehreen Gillani

2026-02-21

Fetching API keys from environment variables and loading necessary libraries for data retrieval and processing.

```
# Load environment variables
import os
from dotenv import load_dotenv
from fredapi import Fred
import pandas as pd

# Load from .env file
load_dotenv()

# Get API keys from environment
FRED_API_KEY = os.getenv("FRED_API_KEY")
BLS_API_KEY = os.getenv("BLS_API_KEY")

# Initialize FRED
fred = Fred(api_key=FRED_API_KEY)

print("API keys loaded successfully!")
```

API keys loaded successfully!

Importing necessary libraries for data retrieval, processing, and analysis.

```
import pandas as pd
import numpy as np
from fredapi import Fred
import requests
import json
```

```

from datetime import datetime
import time

print("="*60)
print("FETCHING ECONOMIC DATA (1999-2024)")
print("="*60)

# Initialize FRED
fred = Fred(api_key=FRED_API_KEY)

# 1. Get Federal Funds Rate from FRED
print("\n1. Fetching Federal Funds Rate from FRED...")
fed_funds = fred.get_series('FEDFUNDS',
                            observation_start='1999-01-01',
                            observation_end='2024-12-31')
fed_df = pd.DataFrame(fed_funds, columns=['fed_funds_rate'])
fed_df.index.name = 'date'
fed_df = fed_df.reset_index()
print(f"    Retrieved {len(fed_df)} records")

# 2. Function to fetch BLS data directly
def fetch_bls_data(series_id, series_name, start_year, end_year, api_key):
    """Fetch data from BLS API"""
    url = 'https://api.bls.gov/publicAPI/v2/timeseries/data/'
    all_data = []

    # BLS API only handles 10 years at a time
    for year_start in range(start_year, end_year, 10):
        year_end = min(year_start + 9, end_year)

        payload = {
            "seriesid": [series_id],
            "startyear": str(year_start),
            "endyear": str(year_end),
            "registrationkey": api_key
        }

        response = requests.post(url, data=json.dumps(payload),
                                headers={'Content-type': 'application/json'})

        if response.status_code == 200:

```

```

        data = response.json()
        if data['status'] == 'REQUEST_SUCCEEDED':
            for series in data['Results']['series']:
                for item in series['data']:
                    date = datetime(int(item['year']), int(item['period'][1:]), 1)
                    all_data.append({
                        'date': date,
                        'series_name': float(item['value'])
                    })

    time.sleep(0.5) # Be nice to the API

    return pd.DataFrame(all_data)

# 3. Fetch CPI data
print("\n2. Fetching CPI data from BLS...")
cpi_df = fetch_bls_data('CUUR0000SA0', 'cpi', 1999, 2024, BLS_API_KEY)
print(f"    Retrieved {len(cpi_df)} records")

# 4. Fetch Unemployment data
print("\n3. Fetching Unemployment data from BLS...")
unemp_df = fetch_bls_data('LNS14000000', 'unemployment', 1999, 2024, BLS_API_KEY)
print(f"    Retrieved {len(unemp_df)} records")

# 5. Merge all data
print("\n4. Merging all datasets...")
economic_data = fed_df.copy()
economic_data = economic_data.merge(cpi_df, on='date', how='left')
economic_data = economic_data.merge(unemp_df, on='date', how='left')
economic_data = economic_data.sort_values('date').reset_index(drop=True)

# 6. Calculate key metrics
print("\n5. Calculating derived metrics...")
economic_data['cpi_yoy'] = economic_data['cpi'].pct_change(12) * 100
economic_data['year'] = economic_data['date'].dt.year
economic_data['month'] = economic_data['date'].dt.month
economic_data['quarter'] = economic_data['date'].dt.quarter
economic_data['decade'] = (economic_data['year'] // 10) * 10

# 7. Calculate mandate performance metrics
economic_data['inflation_goal_met'] = abs(economic_data['cpi_yoy'] - 2) <= 1
economic_data['unemployment_goal_met'] = economic_data['unemployment'] <= 5

```

```

economic_data['both_goals_met'] = economic_data['inflation_goal_met'] & economic_data['unemp
# 8. Add recession indicators for context (optional but helpful)
# NBER recession dates
recession_periods = [
    ('2001-03-01', '2001-11-01'),
    ('2007-12-01', '2009-06-01'),
    ('2020-02-01', '2020-04-01')
]

economic_data['recession'] = 0
for start, end in recession_periods:
    mask = (economic_data['date'] >= start) & (economic_data['date'] <= end)
    economic_data.loc[mask, 'recession'] = 1

# 9. Save to a single CSV file for R
print("\n6. Saving combined data...")
output_file = 'fed_mandate_complete_data.csv'
economic_data.to_csv(output_file, index=False)
print(f"    Data saved to '{output_file}'")
print(f"    Total records: {len(economic_data)}")
print(f"    Date range: {economic_data['date'].min()} to {economic_data['date'].max()}")
print(f"    Columns: {list(economic_data.columns)}")

# 10. Show summary
print("\n" + "="*60)
print("DATA SUMMARY")
print("="*60)
print(economic_data[['fed_funds_rate', 'cpi', 'unemployment', 'cpi_yoy']].describe())

print("\n Python data fetching complete! Now you can use this file in R.")

```

```

=====
FETCHING ECONOMIC DATA (1999-2024)
=====

```

1. Fetching Federal Funds Rate from FRED...

```

/Users/mehreen.gillaniicloud.com/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35:
warnings.warn(

```

Retrieved 312 records

2. Fetching CPI data from BLS...

Retrieved 312 records

3. Fetching Unemployment data from BLS...

Retrieved 312 records

4. Merging all datasets...

5. Calculating derived metrics...

6. Saving combined data...

Data saved to 'fed_mandate_complete_data.csv'

Total records: 312

Date range: 1999-01-01 00:00:00 to 2024-12-01 00:00:00

Columns: ['date', 'fed_funds_rate', 'cpi', 'unemployment', 'cpi_yoy', 'year', 'month', '']

=====

DATA SUMMARY

=====

	fed_funds_rate	cpi	unemployment	cpi_yoy
count	312.000000	312.000000	312.000000	300.000000
mean	2.036923	227.486141	5.638141	2.579248
std	2.068696	39.367210	1.944252	1.770952
min	0.050000	164.300000	3.400000	-2.097161
25%	0.150000	195.200000	4.200000	1.572142
50%	1.260000	226.777000	5.000000	2.293935
75%	4.115000	251.619000	6.225000	3.358685
max	6.540000	315.664000	14.800000	9.059758

Python data fetching complete! Now you can use this file in R.